

机器学习实验指导手册

Python 基础先导实验

一、实验总目标

- 1. 温故并熟练掌握 Python 原生核心语法，包括数据类型、容器结构、流程控制、函数定义、文件操作等核心知识点
- 2. 理解机器学习任务的全流程编程范式，建立**数据-模型-评估**的核心编程逻辑
- 3. 掌握用原生 Python 实现机器学习场景下的基础操作，包括数据集构建、数据预处理、算法逻辑封装、结果评估等
- 4. 培养规范的 Python 代码编写习惯，为后续核心算法实验做好编程能力铺垫

二、实验预备原理

1. 机器学习任务的基础编程流程

标准机器学习任务的编程全流程为：**数据集构建与加载** → **数据预处理与特征工程** → **算法/模型逻辑实现** → **模型预测与结果评估** → **结果可视化与分析**。本实验将围绕该流程，用原生 Python 实现每一个环节的核心操作。

2. Python 原生语法与机器学习场景的映射

Python 语法元素	机器学习场景对应应用
int/float 数值类型	特征值、标签值、模型参数的存储与计算
list 列表/嵌套列表	单条样本、特征向量、标签向量、完整数据集的存储
tuple 元组	不可变的样本属性、算法超参数集合

dict 字典	数据集元信息、超参数、评估结果的结构化存储
for/while 循环、if 条件判断	样本遍历、参数迭代优化、样本正负例筛选等核心逻辑
函数定义与封装	算法模块、预处理函数、评估指标的模块化复用
文件读写	本地数据集加载、实验结果存储

3. 核心概念的原生 Python 表示

- 样本 (Sample): 一条带标签的观测数据, 格式为[特征列表, 标签], 示例: `[[3.2, 1.4, 5.1], 0]`
- 特征 (Feature): 样本的属性值, 为列表中的数值/枚举值, 示例: `[3.2, 1.4, 5.1]`
- 标签 (Label): 样本的目标值, 分类任务为离散值, 回归任务为连续值
- 数据集 (Dataset): 多条样本组成的嵌套列表, 示例: `[[[3.2,1.4],0], [5.1,2.3],1], ...]`

三、分模块实验步骤与任务

模块一：Python 基础语法与机器学习数据结构实现

建议上机时长：60 分钟

1. 模块实验目的

1. 熟练掌握 Python 列表、嵌套列表、字典等容器结构在机器学习数据存储中的应用
2. 掌握循环、条件判断等流程控制语句对数据集的遍历与筛选操作
3. 能够用原生 Python 完成数据集的基础统计与特征提取

2. 实验步骤与代码示例

Python

```

# 示例 1: 构建分类任务原生数据集 (2 特征、2 分类)
# 单条样本格式: [特征列表, 标签]
dataset = [
    [[5.1, 3.5], 0],
    [[4.9, 3.0], 0],
    [[4.7, 3.2], 0],
    [[4.6, 3.1], 0],
    [[7.0, 3.2], 1],
    [[6.4, 3.2], 1],
    [[6.9, 3.1], 1],
    [[5.5, 2.3], 1]
]

# 示例 2: 提取特征矩阵与标签向量
features = [sample[0] for sample in dataset]
labels = [sample[1] for sample in dataset]
print("特征矩阵: ", features)
print("标签向量: ", labels)

# 示例 3: 数据集基础统计
n_samples = len(dataset)
n_features = len(features[0])
n_pos = sum(1 for label in labels if label == 1)
n_neg = n_samples - n_pos
print(f"样本总数: {n_samples}, 特征维度: {n_features}")
print(f"正例数量: {n_pos}, 负例数量: {n_neg}")

# 示例 4: 特征统计函数封装
def calc_feature_stats(features):
    n_features = len(features[0])
    stats = []
    for i in range(n_features):
        feature_values = [sample[i] for sample in features]
        mean_val = sum(feature_values) / len(feature_values)
        max_val = max(feature_values)
        min_val = min(feature_values)
        stats.append({
            "特征索引": i,
            "均值": mean_val,
            "最大值": max_val,
            "最小值": min_val
        })
    return stats

```

```
feature_stats = calc_feature_stats(features)
for stat in feature_stats:
    print(stat)
```

3. 模块实验任务

必做任务（40 分钟）

1. 基于上述示例，构建一个包含 10 条样本、3 个特征、2 分类的原生 Python 数据集，要求正负例样本各 5 条；
2. 编写代码完成以下操作：
 - (1) 提取数据集的特征矩阵与标签向量，打印输出；
 - (2) 统计并打印数据集的样本总数、特征维度、正负例占比；
 - (3) 原生 Python 实现函数，计算每个特征的方差，输出每个特征的方差结果；
 - (4) 筛选出第 0 个特征值大于 5 的所有样本，打印输出对应的特征与标签；
3. 用 Python 标准库 `random` 实现样本打乱，将数据集按 7:3 的比例划分为训练集与测试集，输出训练集和测试集的样本数量。

扩展任务（20 分钟）

1. 用字典结构重构数据集，每条样本包含样本 ID「特征列表」「标签」三个键值对，实现上述所有统计与筛选操作；
2. 实现按指定特征值对数据集进行升序/降序排序，输出排序后的数据集。

模块二：机器学习数据预处理的原生 Python 实现

建议上机时长：60 分钟

1. 模块实验目的

1. 理解机器学习中数据预处理的核心作用与常用方法
2. 用原生 Python 实现标准化、归一化、缺失值处理、特征编码等核心预处理操作
3. 掌握预处理函数的封装与复用方法

2. 核心原理

1. 最小-最大归一化：将特征缩放到[0,1]区间，公式：
$$x' = \frac{x - x_{min}}{x_{max} - x_{min}}$$

2. Z-Score 标准化：将特征缩放为均值 0、方差 1 的分布，公式： $x' = \frac{x-\mu}{\sigma}$ (μ 为特征均值， σ 为特征标准差)
3. 缺失值处理：常用均值/中位数填充、删除缺失样本两种方式
4. 离散特征编码：将分类特征（如“晴”“阴”“雨”）转换为可计算的数值型编码

3. 实验步骤与代码示例

Python

示例 1: 最小-最大归一化原生实现

```
def min_max_scaler(features):
    n_features = len(features[0])
    scaled_features = []
    # 计算每个特征的极值
    feature_minmax = []
    for i in range(n_features):
        values = [sample[i] for sample in features]
        min_val = min(values)
        max_val = max(values)
        feature_minmax.append((min_val, max_val))
    # 逐样本缩放
    for sample in features:
        scaled_sample = []
        for i in range(n_features):
            min_val, max_val = feature_minmax[i]
            scaled_val = 0.0 if max_val == min_val else (sample[i]
- min_val) / (max_val - min_val)
            scaled_sample.append(scaled_val)
        scaled_features.append(scaled_sample)
    return scaled_features, feature_minmax
```

测试

```
features = [[5.1, 3.5], [4.9, 3.0], [7.0, 3.2], [6.4, 3.2]]
scaled_features, minmax_params = min_max_scaler(features)
print("归一化后特征: ", scaled_features)
```

示例 2: 缺失值填充原生实现

```
def fill_missing_value(features, fill_strategy="mean"):
    n_features = len(features[0])
    filled_features = [sample.copy() for sample in features]
    # 计算每个特征的填充值
    fill_values = []
    for i in range(n_features):
```

```

        valid_values = [sample[i] for sample in features if
sample[i] is not None]
        if fill_strategy == "mean":
            fill_val = sum(valid_values) / len(valid_values)
        elif fill_strategy == "median":
            sorted_vals = sorted(valid_values)
            n = len(sorted_vals)
            fill_val = sorted_vals[n//2] if n%2 ==1 else
(sorted_vals[n//2-1]+sorted_vals[n//2])/2
        else:
            raise ValueError("可选填充策略: mean/median")
        fill_values.append(fill_val)
# 执行填充
for sample in filled_features:
    for i in range(n_features):
        if sample[i] is None:
            sample[i] = fill_values[i]
return filled_features, fill_values

# 测试
features_with_nan = [[5.1, None], [4.9, 3.0], [None, 3.2], [6.4,
3.2]]
filled_features, fill_vals = fill_missing_value(features_with_nan,
fill_strategy="mean")
print("填充后特征: ", filled_features)

```

4. 模块实验任务

必做任务（40 分钟）

1. 基于模块一构建的数据集，原生 Python 实现 Z-Score 标准化函数：
 - (1) 输入为特征矩阵，输出为标准化后的特征矩阵、每个特征的均值和标准差；
 - (2) 处理除零错误（特征标准差为 0 时，标准化后值统一为 0）；
 - (3) 对数据集进行标准化，打印标准化前后的特征均值与方差，验证效果。
2. 构建含缺失值的特征矩阵（用 None 表示缺失），实现中位数填充函数，完成缺失值处理，打印填充前后的特征矩阵。
3. 实现离散特征的标签编码函数：输入为离散特征列表（如["Sunny", "Rainy", "Sunny", "Cloudy"]），输出为编码后的数值列表与编码映射字典，完成测试验证。

扩展任务（20 分钟）

1. 实现离散特征的独热编码（One-Hot Encoding）函数，输入为离散特征列表，输出为独热编码矩阵与编码映射字典；
2. 实现预处理流程链式调用：对原始数据依次执行缺失值填充→标准化→数据集划分，封装为一个完整的预处理函数。

模块三：机器学习核心逻辑的函数封装与模块化实现

建议上机时长：70 分钟

1. 模块实验目的

1. 掌握 Python 函数封装、参数传递、返回值处理的规范方法
2. 理解机器学习算法的模块化编程思想，建立“算法可复用、可扩展”的编程习惯
3. 用原生 Python 实现机器学习基础算法核心逻辑，为后续核心实验铺垫

2. 核心原理

本模块通过实现 K 近邻（KNN）分类算法，掌握机器学习算法的模块化实现方法。

KNN 核心逻辑：

1. 计算测试样本与所有训练样本的欧式距离： $d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$
2. 选取距离最近的 K 个训练样本
3. 少数服从多数投票，决定测试样本的分类标签

3. 实验步骤与代码示例

```
Python
import math

# 模块 1：计算欧式距离
def calc_euclidean_distance(x1, x2):
    if len(x1) != len(x2):
        raise ValueError("两个样本的特征维度必须一致")
    squared_sum = 0.0
    for a, b in zip(x1, x2):
        squared_sum += (a - b) ** 2
    return math.sqrt(squared_sum)

# 模块 2：获取 K 个最近邻
def get_k_neighbors(train_set, test_sample, k):
```

```

distances = []
for train_sample in train_set:
    train_features = train_sample[0]
    train_label = train_sample[1]
    dist = calc_euclidean_distance(test_sample,
train_features)
    distances.append((train_sample, dist))
# 按距离升序排序
distances.sort(key=lambda x: x[1])
# 选取前 K 个近邻
neighbors = [item[0] for item in distances[:k]]
return neighbors

# 模块 3: 投票预测标签
def predict(neighbors):
    label_votes = {}
    for neighbor in neighbors:
        label = neighbor[1]
        label_votes[label] = label_votes.get(label, 0) + 1
    # 返回票数最高的标签
    sorted_votes = sorted(label_votes.items(), key=lambda x: x[1],
reverse=True)
    return sorted_votes[0][0]

# 模块 4: KNN 完整流程封装
def knn_classifier(train_set, test_set, k=3):
    predictions = []
    for test_sample in test_set:
        test_features = test_sample[0]
        neighbors = get_k_neighbors(train_set, test_features, k)
        pred_label = predict(neighbors)
        predictions.append(pred_label)
    return predictions

# 测试
train_set = [
    [[5.1, 3.5], 0], [[4.9, 3.0], 0], [[4.7, 3.2], 0],
    [[7.0, 3.2], 1], [[6.4, 3.2], 1], [[6.9, 3.1], 1]
]
test_set = [[[5.0, 3.4], 0], [[6.5, 3.0], 1]]

preds = knn_classifier(train_set, test_set, k=3)
print("预测标签: ", preds)

```



```
print("真实标签: ", [sample[1] for sample in test_set])
```

4. 模块实验任务

必做任务（50 分钟）

1. 补全上述 KNN 算法所有模块，原生 Python 实现，完成代码调试与测试验证；
2. 基于模块一构建的数据集，完成以下操作：
 - (1) 将数据集按 7:3 划分为训练集与测试集；
 - (2) 对训练集和测试集进行标准化预处理；
 - (3) 分别设置 $k=1$ 、 $k=3$ 、 $k=5$ ，调用 KNN 分类器完成预测，输出每组 k 值对应的预测标签与真实标签；
3. 原生 Python 实现曼哈顿距离计算函数（公式： $d(x,y) = \sum_{i=1}^n |x_i - y_i|$ ），替换 KNN 中的欧式距离，完成测试验证。

扩展任务（20 分钟）

1. 实现带权重的 KNN 算法：距离越近的样本投票权重越高（权重为距离的倒数），完成代码实现与测试；
2. 封装 KNN 算法为 Python 类，包含 `__init__`（设置 k 值、距离类型）、`fit`（存储训练集）、`predict`（预测）三个核心方法，完成面向对象编程实现。

模块四：模型评估指标与结果可视化实现

建议上机时长：50 分钟

1. 模块实验目的

1. 理解机器学习分类任务核心评估指标的原理
2. 用原生 Python 实现分类任务核心评估指标的计算
3. 掌握用 matplotlib 实现结果可视化的方法，建立结果分析能力

2. 核心原理

分类任务核心评估指标基于混淆矩阵计算：

- TP（真正例）：真实为正、预测为正的样本数
- TN（真负例）：真实为负、预测为负的样本数

- FP (假正例): 真实为负、预测为正的样本数
- FN (假负例): 真实为正、预测为负的样本数

核心指标公式:

1. 准确率: $Acc = \frac{TP+TN}{TP+TN+FP+FN}$
2. 精确率: $Precision = \frac{TP}{TP+FP}$
3. 召回率: $Recall = \frac{TP}{TP+FN}$
4. F1 值: $F1 = 2 * \frac{Precision * Recall}{Precision + Recall}$

3. 实验步骤与代码示例

Python

示例 1: 分类评估指标原生实现

```
def calc_classification_metrics(y_true, y_pred, pos_label=1):
    # 计算混淆矩阵元素
    TP = TN = FP = FN = 0
    for true, pred in zip(y_true, y_pred):
        if true == pos_label and pred == pos_label:
            TP += 1
        elif true != pos_label and pred != pos_label:
            TN += 1
        elif true != pos_label and pred == pos_label:
            FP += 1
        elif true == pos_label and pred != pos_label:
            FN += 1
    # 计算评估指标
    total = TP + TN + FP + FN
    accuracy = (TP + TN) / total if total != 0 else 0.0
    precision = TP / (TP + FP) if (TP + FP) != 0 else 0.0
    recall = TP / (TP + FN) if (TP + FN) != 0 else 0.0
    f1 = 2 * precision * recall / (precision + recall) if (precision + recall) != 0 else 0.0
    # 返回结果
    return {
        "混淆矩阵": {"TP": TP, "TN": TN, "FP": FP, "FN": FN},
        "准确率": accuracy,
        "精确率": precision,
        "召回率": recall,
        "F1 值": f1
    }
```

```

# 测试
y_true = [0, 1, 0, 1, 0, 1]
y_pred = [0, 1, 1, 1, 0, 0]
metrics = calc_classification_metrics(y_true, y_pred)
for key, value in metrics.items():
    print(f"{key}: {value}")

# 示例 2: 预测结果可视化
import matplotlib.pyplot as plt

def plot_predict_result(y_true, y_pred):
    plt.figure(figsize=(8, 4))
    x = range(len(y_true))
    plt.scatter(x, y_true, label="真实标签", color="blue", s=60)
    plt.scatter(x, y_pred, label="预测标签", color="red",
marker="x", s=60)
    plt.xlabel("样本索引")
    plt.ylabel("标签")
    plt.title("真实标签与预测标签对比")
    plt.yticks([0, 1])
    plt.legend()
    plt.grid(True, linestyle="--", alpha=0.7)
    plt.show()

plot_predict_result(y_true, y_pred)

```

4. 模块实验任务

必做任务（30 分钟）

1. 基于模块三的 KNN 预测结果，调用上述评估函数，分别计算 $k=1$ 、 $k=3$ 、 $k=5$ 时的准确率、精确率、召回率、F1 值，打印所有结果，分析 k 值对模型效果的影响；
2. 编写代码，绘制不同 k 值对应的准确率变化折线图；
3. 绘制测试集样本的真实标签与预测标签对比散点图，完成结果分析。

扩展任务（20 分钟）

1. 原生 Python 实现混淆矩阵绘制函数，用 matplotlib 实现热力图可视化；
2. 实现回归任务核心评估指标：均方误差（MSE）、平均绝对误差（MAE）、决定系数 R^2 的原生 Python 实现，完成测试验证。

四、整体实验要求

必做总任务

1. 完成上述四个模块的所有必做任务，提交完整的 Python 代码文件，代码需添加规范注释，保证可直接运行；
2. 撰写实验报告，包含以下内容：
 - (1) 每个模块的代码实现与运行结果截图；
 - (2) KNN 算法中 k 值变化对模型效果的影响分析；
 - (3) 对机器学习全流程编程范式的理解与总结；
 - (4) 实验中遇到的问题与解决方法；
3. 完成代码单元测试，用断言验证每个函数模块的功能正确性。

扩展总任务

1. 基于原生 Python，实现单变量线性回归模型，包括梯度下降参数求解、预测、MSE 评估，为后续核心实验做准备；
2. 用 Python 标准库 `csv` 模块加载外部 CSV 格式数据集，完成预处理→模型训练→预测→评估全流程；
3. 分析 KNN 算法的时间复杂度，对比不同样本数量、特征维度下的算法运行效率。

五、思考题

1. 机器学习任务中，为什么要先对数据进行预处理？不做预处理会对模型效果产生什么影响？
2. KNN 算法中，k 值的大小对模型的欠拟合与过拟合有什么影响？
3. 原生 Python 实现的算法与调用第三方库（如 `sklearn`）相比，有什么优势与劣势？
4. 模块化编程与面向对象编程，在机器学习算法实现中各有什么适用场景？

六、实验注意事项

1. 实验全程优先使用 Python 原生语法与标准库实现，禁止使用 PyTorch、

TensorFlow 等深度学习框架，核心逻辑禁止直接调用 sklearn 等第三方机器学习库的封装函数；

2. 代码编写需遵循 Python PEP8 规范，添加清晰的注释，保证代码的可读性与可维护性；
3. 实验过程中需分步调试代码，先保证每个函数模块的正确性，再进行整体流程的串联；
4. 实验报告需真实记录实验过程与结果，独立完成，**严禁抄袭获交 AI 自动生成**。